



COMP3026E Distributed Systems

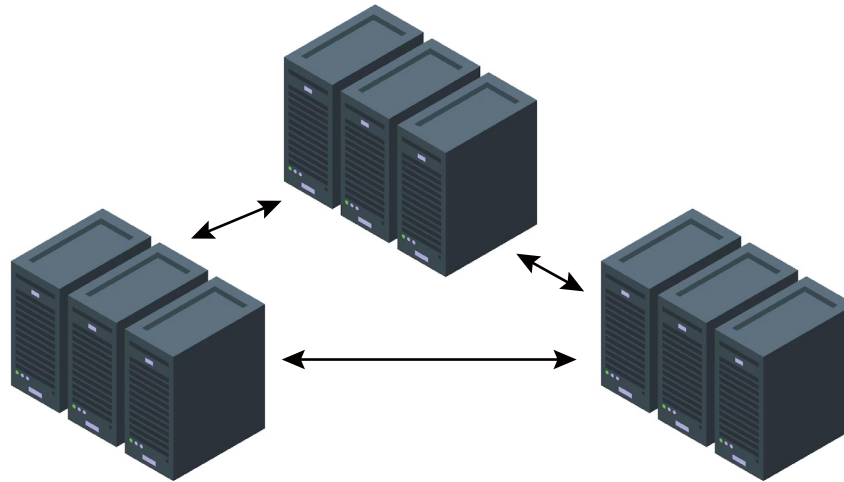
2. Network and RPC

James Jianqiao Yu

Harbin Institute of Technology, Shenzhen

Distributed Systems, What?

- A distributed system is a collection of independent computers that appear to the users of the system as a single computer.
 - Multiple computers
 - **Connected by a network**
 - Doing a common task

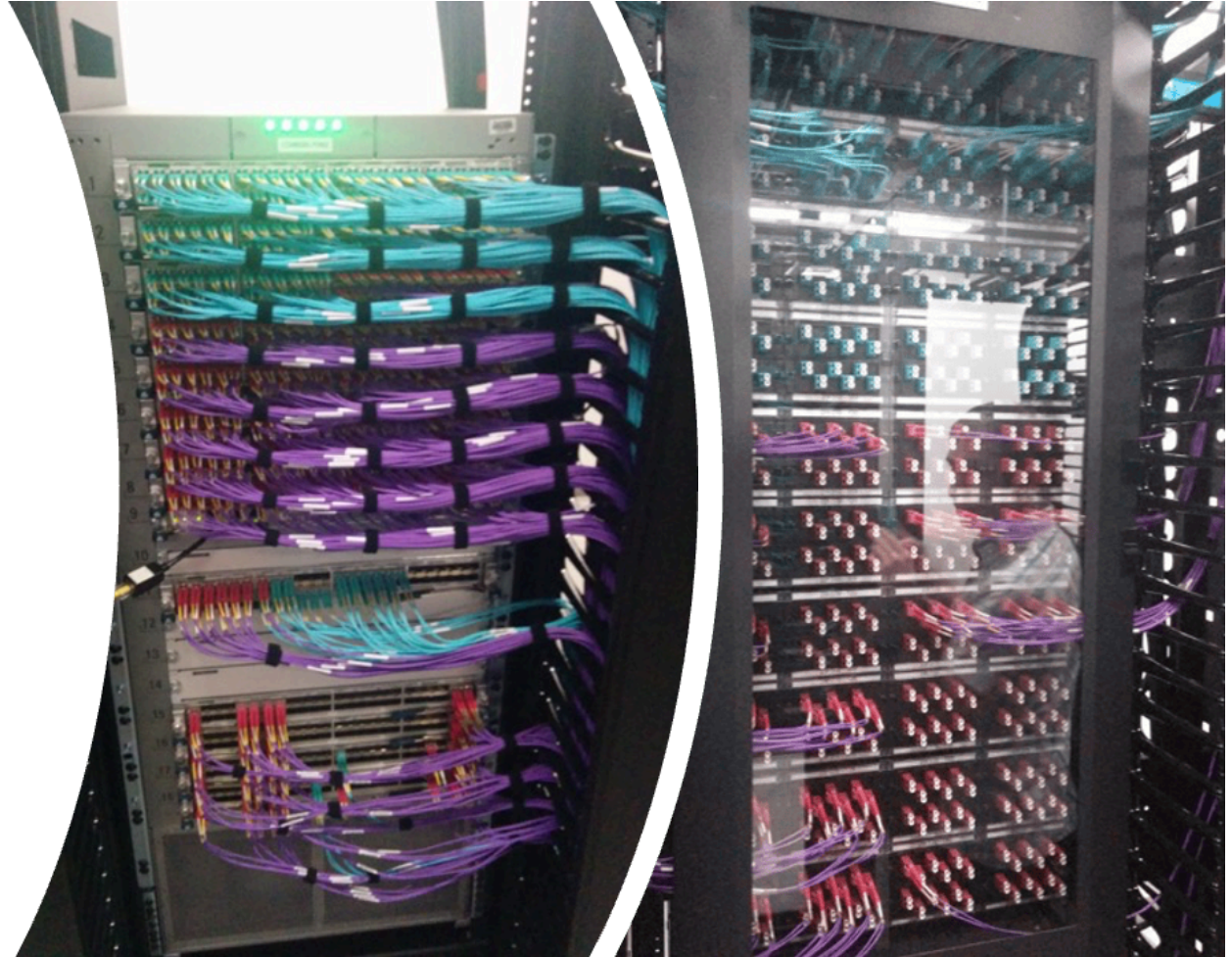


Facebook Uses Distributed Systems



Facebook Uses Distributed Systems

- Facebook has 100+ data centers around the world, connected by a private **fiber network**.
- Each data center has 100,000+ servers, connected by a high-speed **local network**.



Today's Outline



- How can processes on different cooperating computers communicate with each other over the network?
1. Network Communication
 2. Remote Procedure Call (RPC)

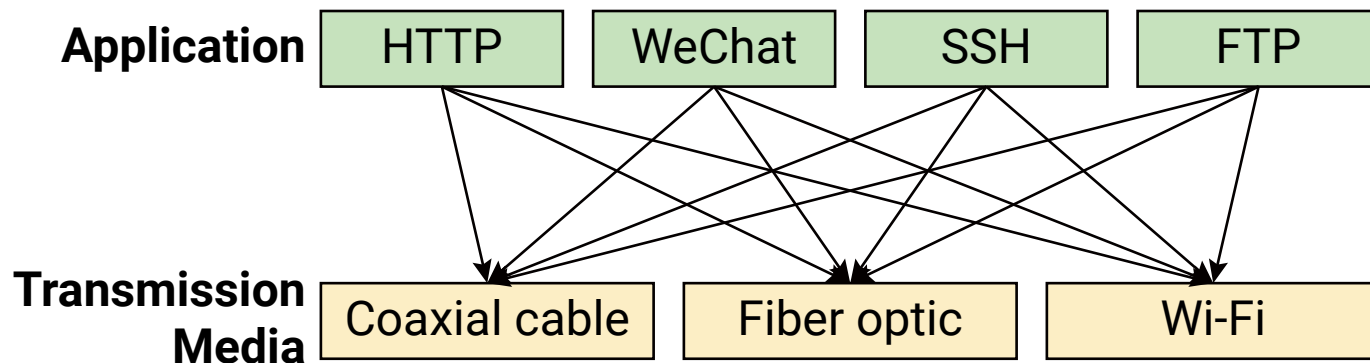
The Problem of Communication

- Process on Host A wants to talk to process on Host B
- A and B must agree on the meaning of the bits being sent and received at many different levels, including:
 - How many volts is a 0 bit, a 1 bit?
 - How does receiver know which is the last bit?
 - How many bits long is a number?





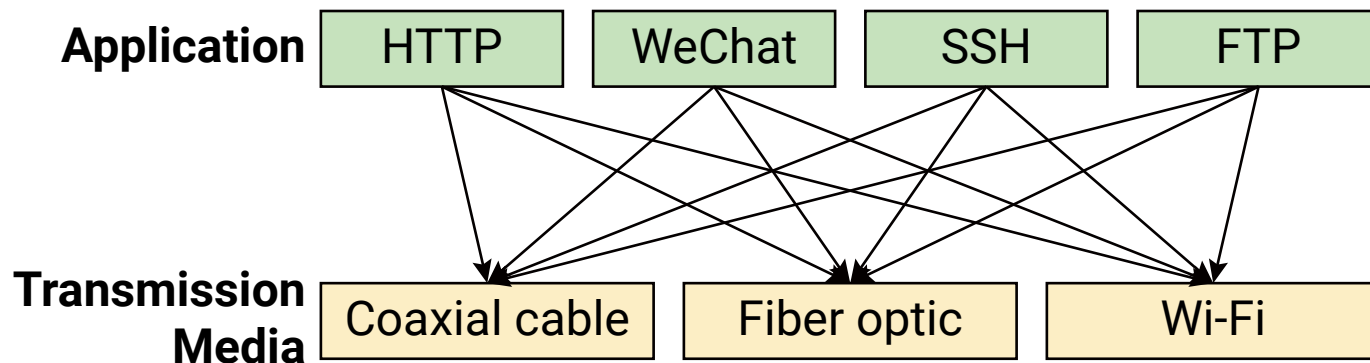
The Problem of Communication



- **Re-implement every application** for every new transmission medium?
- **Change every application** on any change to a transmission medium?

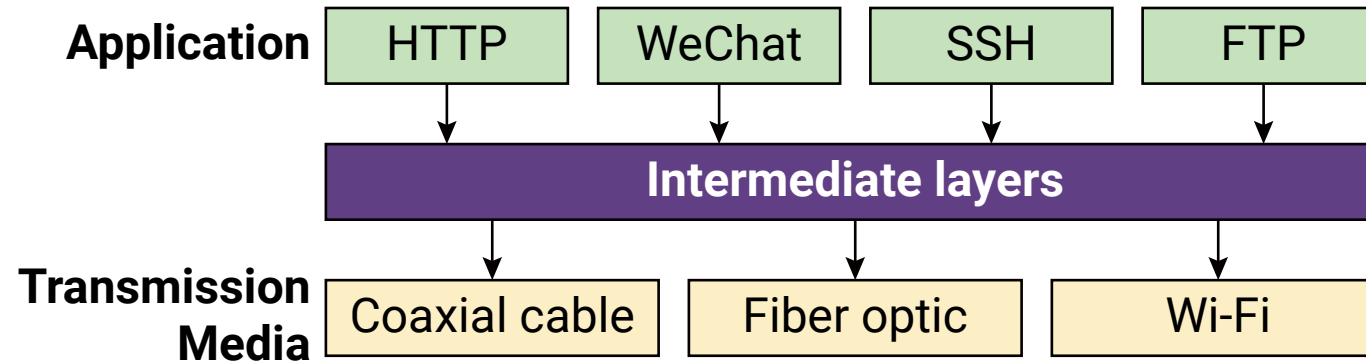


The Problem of Communication



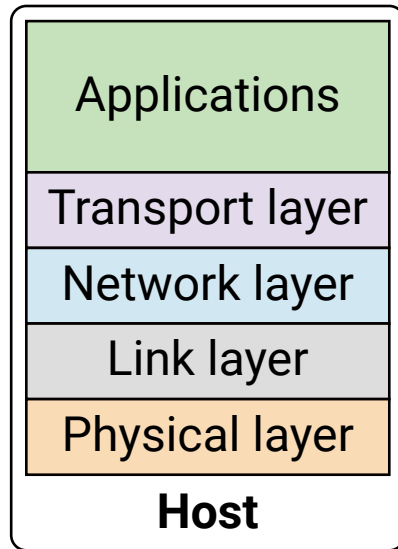
- **Re-implement every application** for every new transmission medium?
- **Change every application** on any change to a transmission medium?
 - No! But how does the Internet design avoid this?

Solution: Layering



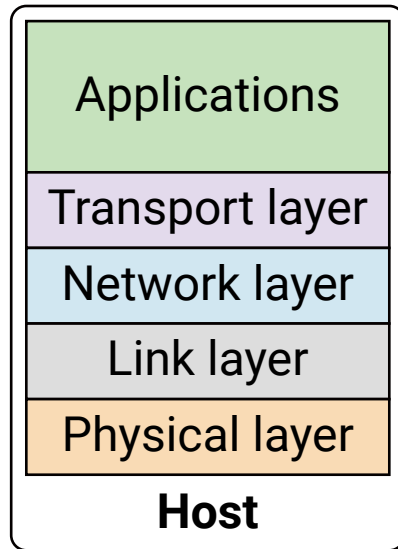
- Intermediate **layers** provide set of abstractions for applications and media
- New apps or media need only implement for intermediate layer's interface

Layering in the Internet



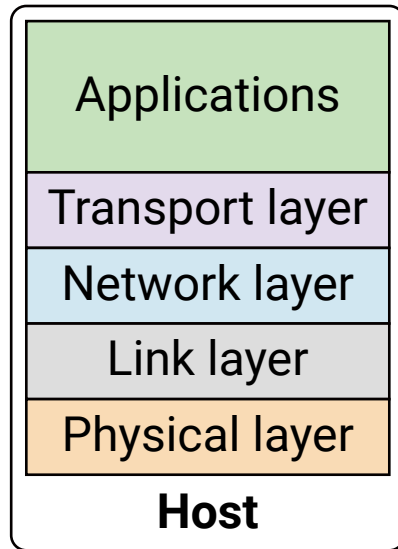
- **Transport:** Provide end-to-end communication between processes on different hosts

Layering in the Internet



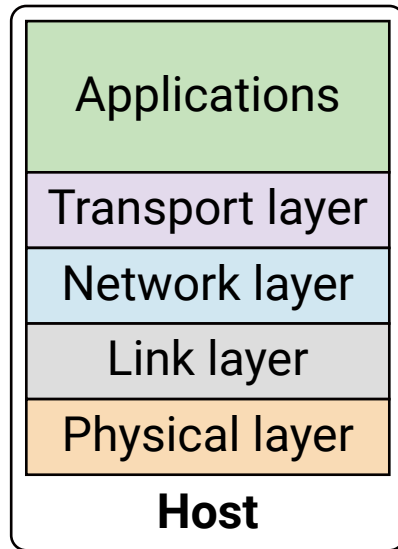
- **Transport:** Provide end-to-end communication between processes on different hosts
- **Network:** Deliver packets to destinations on other (heterogeneous) networks

Layering in the Internet



- **Transport**: Provide end-to-end communication between processes on different hosts
- **Network**: Deliver packets to destinations on other (heterogeneous) networks
- **Link**: Enables end hosts to exchange atomic messages with each other

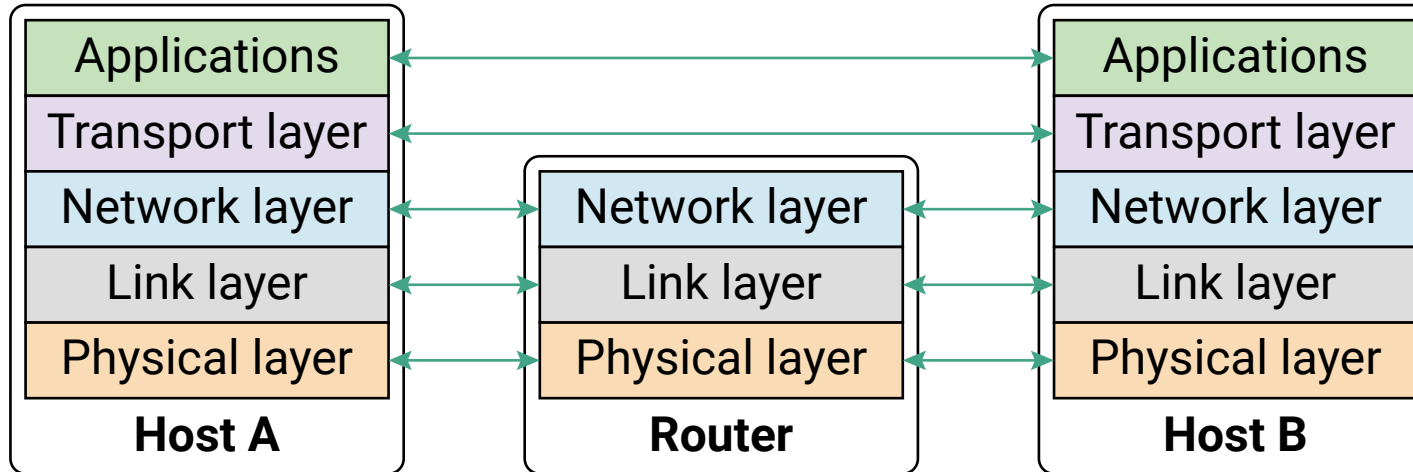
Layering in the Internet



- **Transport**: Provide end-to-end communication between processes on different hosts
- **Network**: Deliver packets to destinations on other (heterogeneous) networks
- **Link**: Enables end hosts to exchange atomic messages with each other
- **Physical**: Moves bits between two hosts connected by a physical link

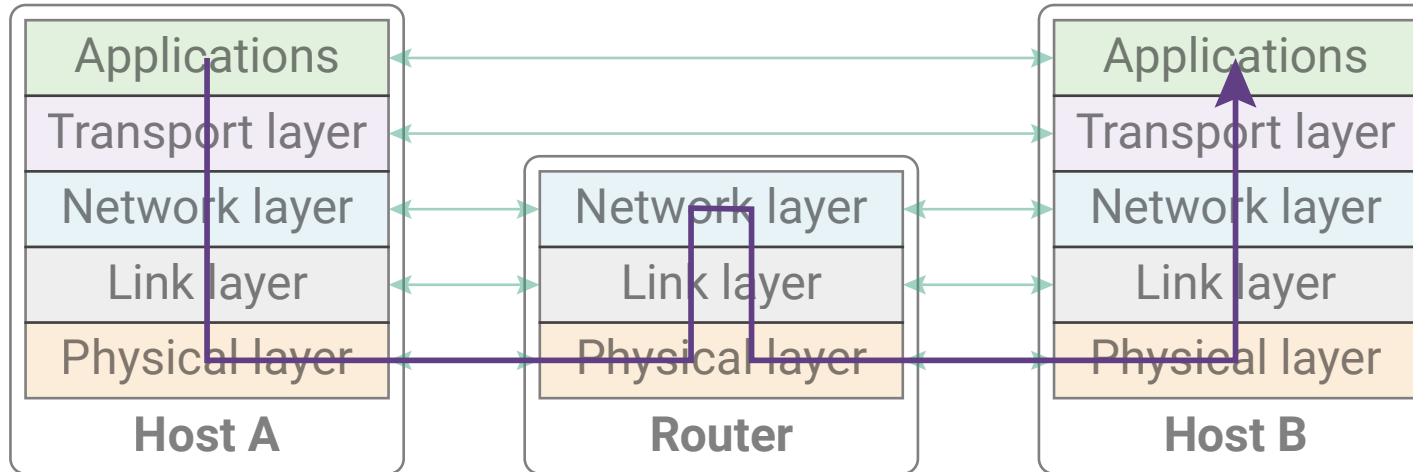
Logical Communication Between Layers

- How to forge agreement on meaning of bits exchanged b/w hosts?
- **Protocol**: Rules that govern format, contents, and meaning of bits
 - Each layer on a host interacts with its peer host's corresponding layer via the **protocol interface**



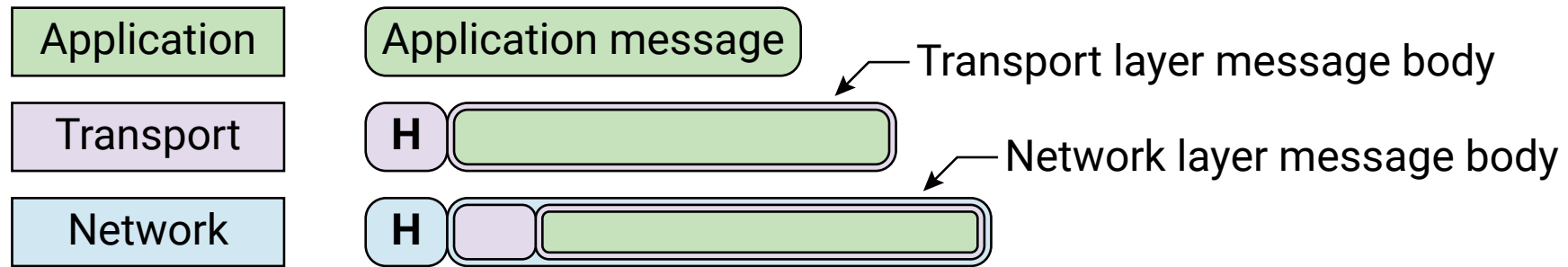
Physical Communication

- Communication goes down to the **physical network**
- Then from network peer to peer
- Then up to the relevant application



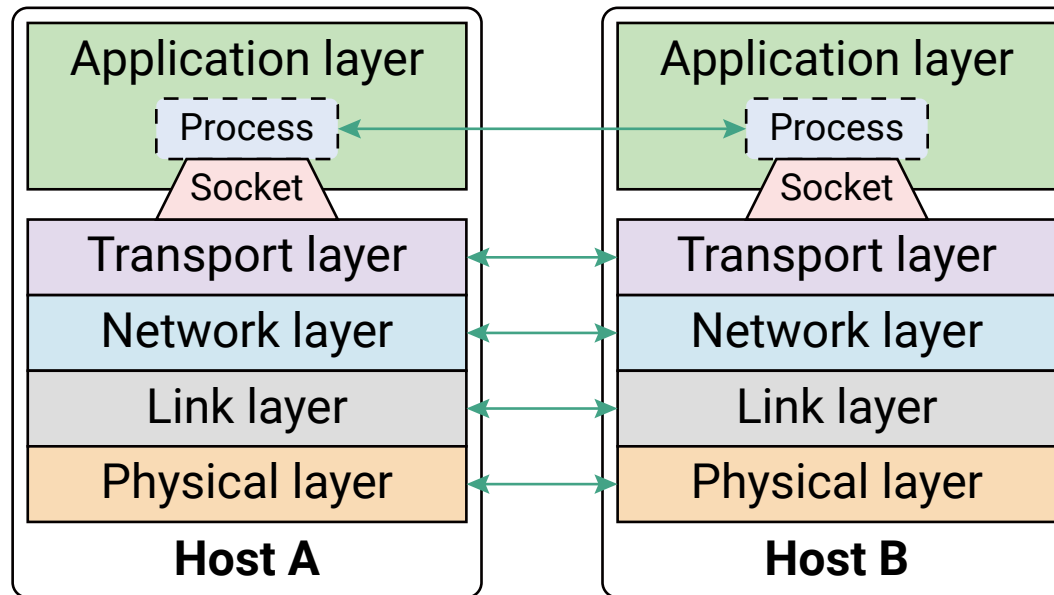
Communication Between Peers

- How do peer protocols coordinate with each other?
- Layer attaches its own **header** (H) to communicate with peer
 - Higher layers' headers, data **encapsulated** inside message
 - Lower layers don't generally inspect higher layers' headers



Network Socket-based Communication

- Socket: The interface the OS provides to the network
 - Provides inter-process explicit message exchange
- Can build distributed systems atop sockets: `send()`, `recv()`
 - e.g.: `put(key, value)` -> message



Network Socket-based Communication

```
1 // Create a socket for the client
2 if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
3     perror("Socket creation");
4     exit(2);
5 }
6
7 // Set server address and port
8 memset(&servaddr, 0, sizeof(servaddr));
9 servaddr.sin_family = AF_INET;
10 servaddr.sin_addr.s_addr = inet_addr(argv[1]);
11 servaddr.sin_port = htons(SERV_PORT); // to big-endian
12
13 // Establish TCP connection
14 if (connect(sockfd, (struct sockaddr *)&servaddr,
15     sizeof(servaddr)) < 0) {
16     perror("Connect to server");
```



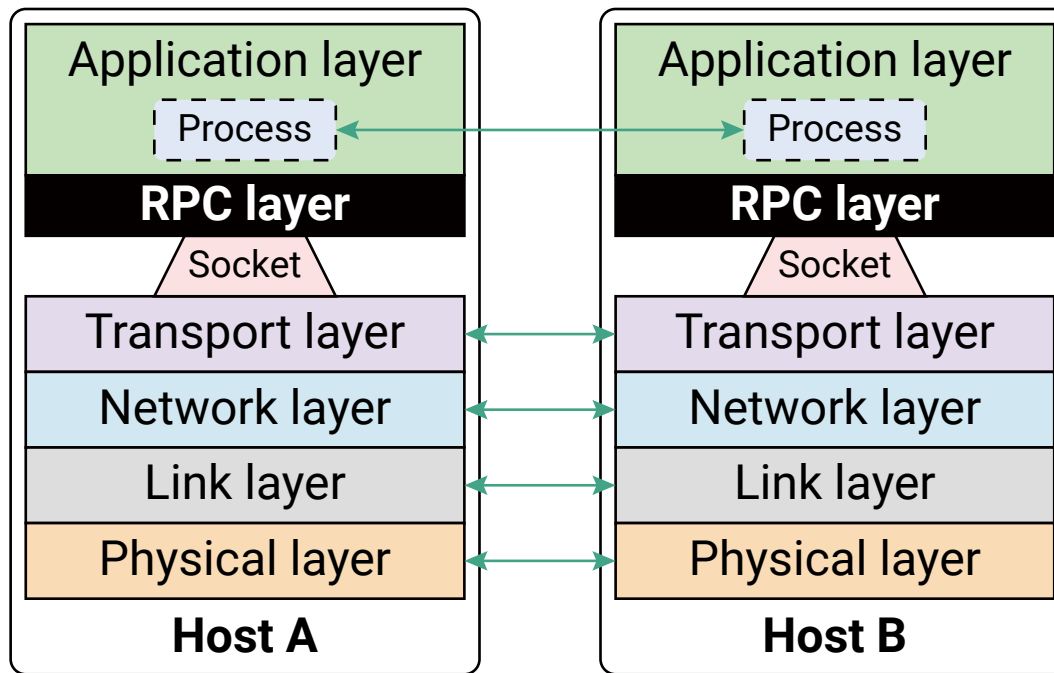
```
17  exit(3);  
18  }  
19  
20  // Transmit the data over the TCP connection  
21  send(sockfd, buf, strlen(buf), 0);
```

- What does this code do?
 - Create a TCP socket for the client process: `sockfd`
 - Fill in the server's IPv4 address and port (in network byte order): `servaddr`
 - Connect to the server using TCP: `connect()`
 - Send the contents of `buf` to the server over the established connection: `send()`

Socket Programming: Still Not Great

- Lots for the programmer to deal with every time
 - How to separate **different requests** on the **same connection**?
 - How to **write** bytes to the network / **read** bytes from the network?
 - What if Host A's process is written in Python and Host B's process is in C++?
 - What to do with those bytes?
- Still pretty painful have to worry a lot about the network

Solution: Another Layer!





Why RPC?



- The typical programmer is trained to write single-threaded code that runs in one place

Objective Easy-to-program network communication that makes client-server communication seem transparent.

- Retains the “feel” of writing centralized code
- Programmer needn’t think (much) about the network

Everyone Uses RPCs



- The programming assignments of this course use RPC
- Google gRPC
- Facebook/Apache Thrift
- Apache Avro
- ...

What's the Goal of RPC?

- Within a single program, running in a single process, recall the well-known notion of a **procedure call**:
 - **Caller** pushes arguments onto stack,
 - jumps to address of **callee** function
 - **Callee** reads arguments from stack,
 - executes, puts return value in register,
 - returns to next instruction in **caller**

What's the Goal of RPC?

- Within a single program, running in a single process, recall the well-known notion of a **procedure call**:
 - **Caller** pushes arguments onto stack,
 - jumps to address of **callee** function
 - **Callee** reads arguments from stack,
 - executes, puts return value in register,
 - returns to next instruction in **caller**

RPC's Goal Make communication appear like a local procedure call: way less painful than sockets.



1. Heterogeneity

- Client needs to rendezvous with the server
- Server must dispatch to the required function
- What if server is different type of machine?



1. Heterogeneity

- Client needs to rendezvous with the server
- Server must dispatch to the required function
- What if server is different type of machine?

2. Failure

- What if messages get **dropped**?
- What if client, server, or network **fails**?



1. Heterogeneity

- Client needs to rendezvous with the server
- Server must dispatch to the required function
- What if server is different type of machine?

2. Failure

- What if messages get **dropped**?
- What if client, server, or network **fails**?

3. Performance

- Procedure call takes ≈ 10 cycles ≈ 3 ns
- RPC in a data center takes ≈ 10 μ s ($10^3 \times$ slower)
- In the wide area, typically ≈ 10 – 100 ms ($10^6 \times$ slower)

Problem: Heterogeneity in Data Representation

- Not an issue for local procedure calls
- For a remote procedure call, a remote machine may:
 - Run process written in a **different language**
 - Represent data types using **different sizes** (e.g., 32 bit vs. 64 bit integers)
 - Use a **different byte ordering** (endianness)
 - Represent floating point numbers **differently**
 - Have **different data alignment** requirements
 - e.g., 4-byte type begins only on 4-byte memory boundary



Solution: Interface Description Language (IDL)



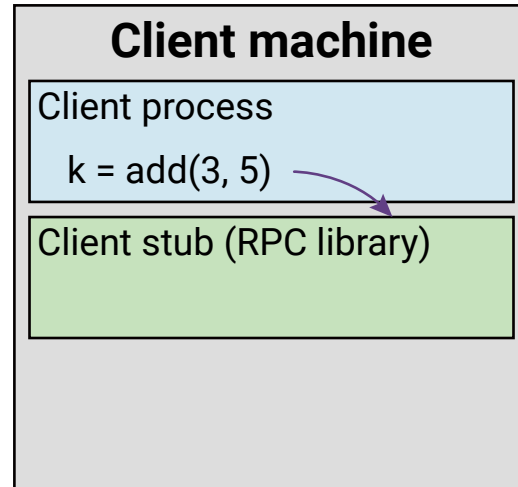
- Mechanism to pass procedure parameters and return values in a machine-independent way
- Programmer may write an **interface description** in the IDL
 - Defines API for procedure calls: names, parameter/return types
- Then runs an **IDL compiler** which generates:
 - Code to **marshal** (convert) native data types into machine-independent byte streams (and vice-versa, called **unmarshaling**)
 - Client stub: Forwards local procedure call as a request to server
 - Server stub: Dispatches RPC to its implementation



A Day in the Life of an RPC

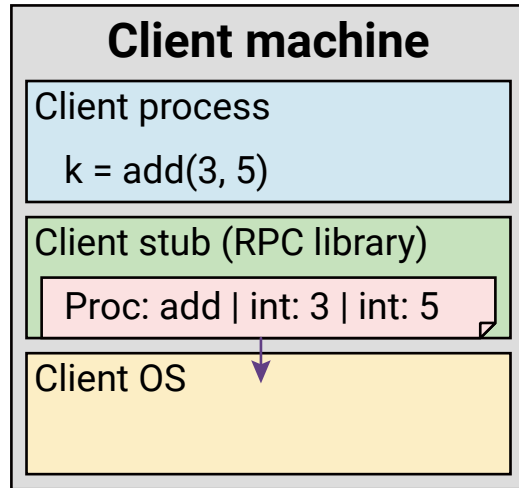


1. Client calls stub function (pushes parameters onto stack)



A Day in the Life of an RPC

1. Client calls stub function (pushes parameters onto stack)
2. Stub marshals parameters to a network message

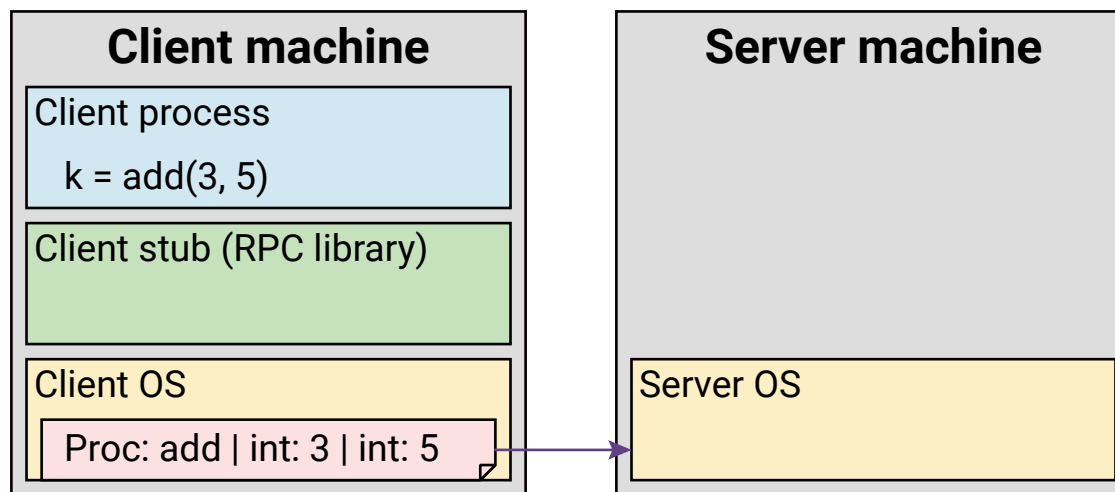




A Day in the Life of an RPC

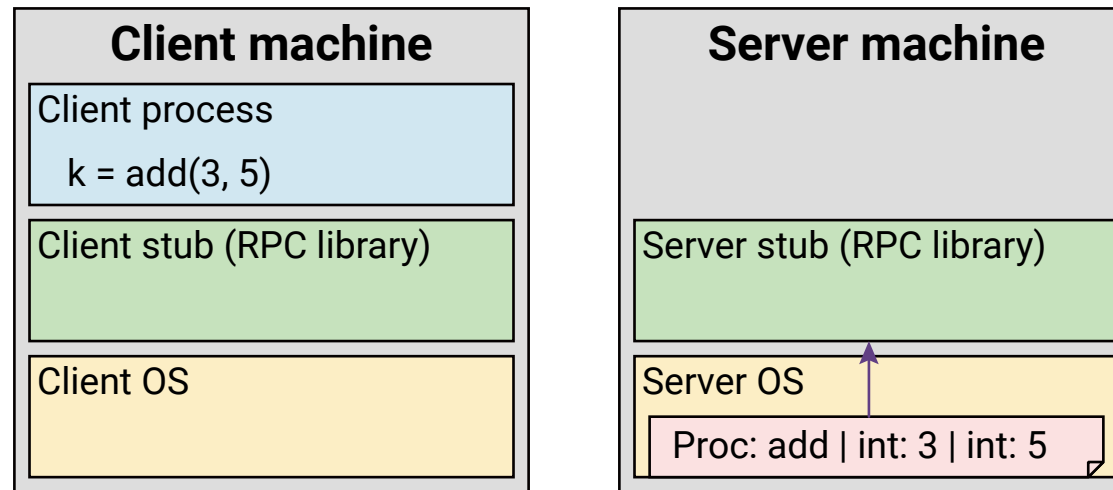


2. Stub marshals parameters to a network message
3. OS sends a network message to the server



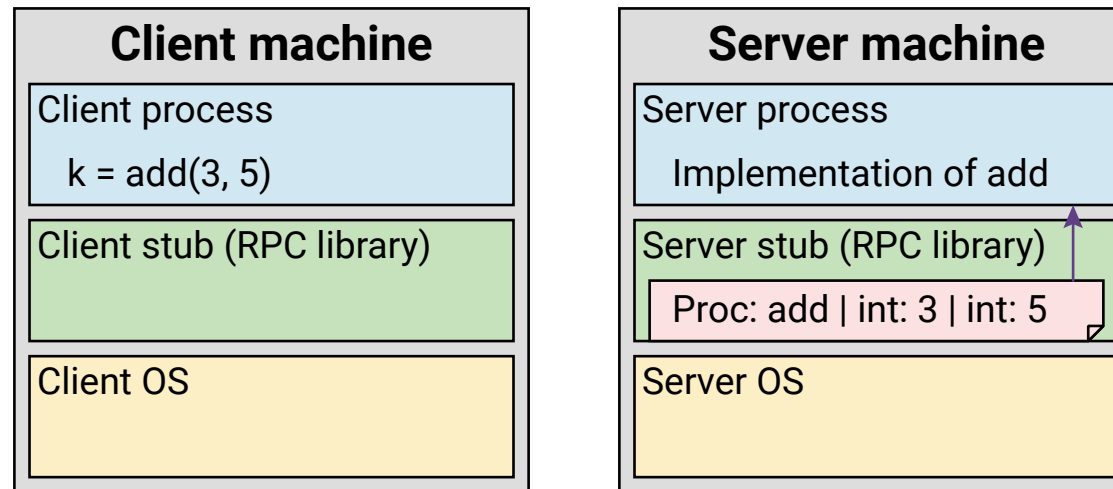
A Day in the Life of an RPC

3. OS sends a network message to the server
4. Server OS receives message, sends it up to stub



A Day in the Life of an RPC

4. Server OS receives message, sends it up to stub
5. Server stub unmarshals params, calls server function

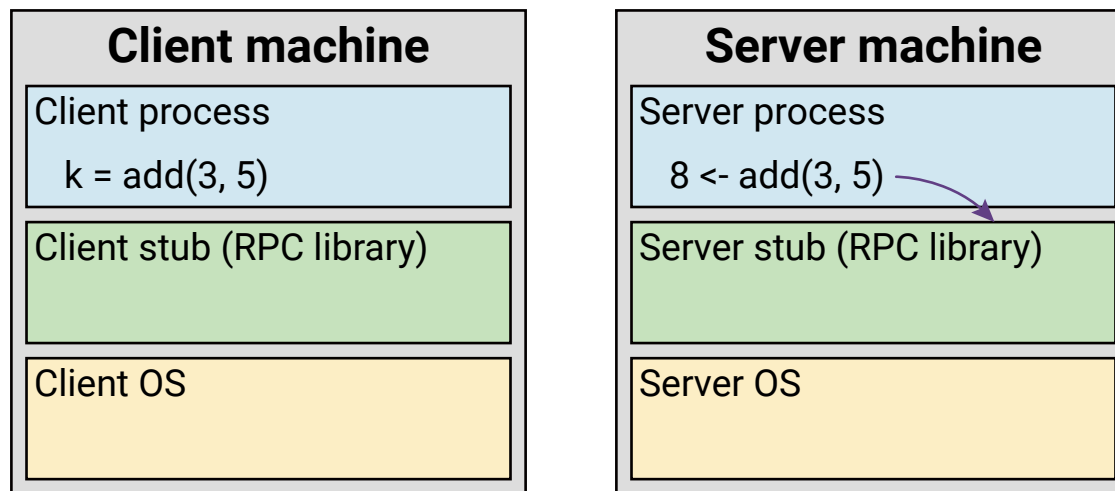




A Day in the Life of an RPC



5. Server stub unmarshals params, calls server function
6. Server function runs, returns a value

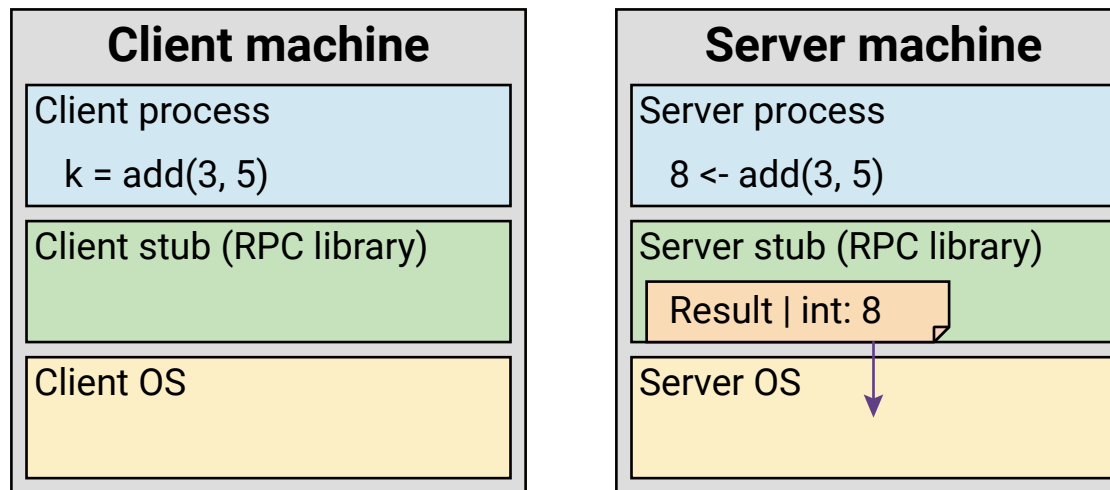




A Day in the Life of an RPC



6. Server function runs, returns a value
7. Server stub marshals the return value, sends message

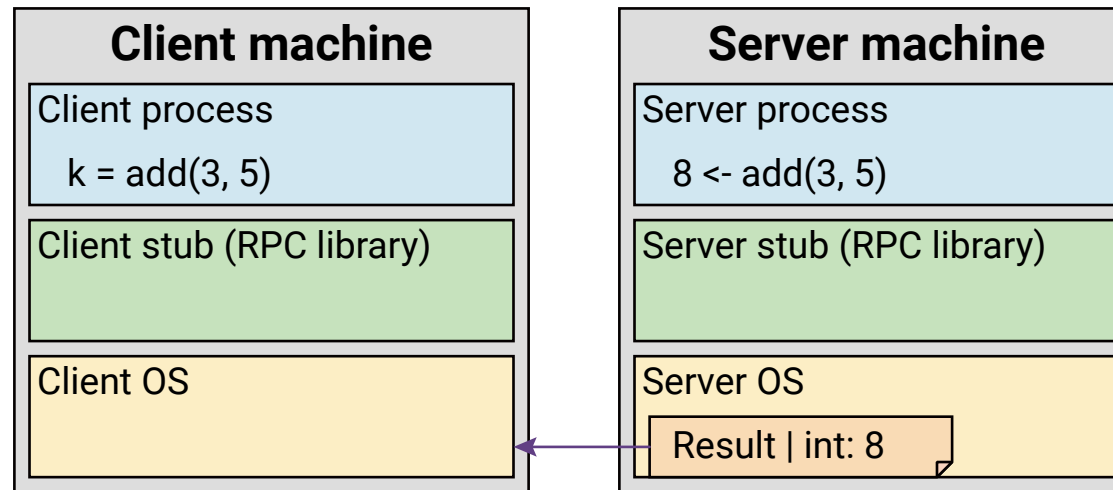




A Day in the Life of an RPC



7. Server stub marshals the return value, sends message
8. Server OS sends the reply back across the network

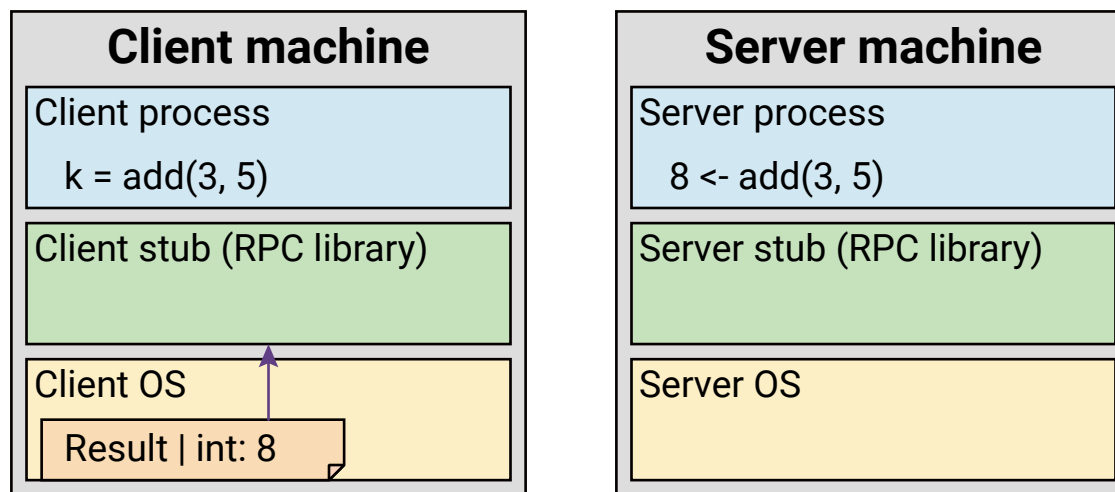




A Day in the Life of an RPC



8. Server OS sends the reply back across the network
9. Client OS receives the reply and passes up to stub

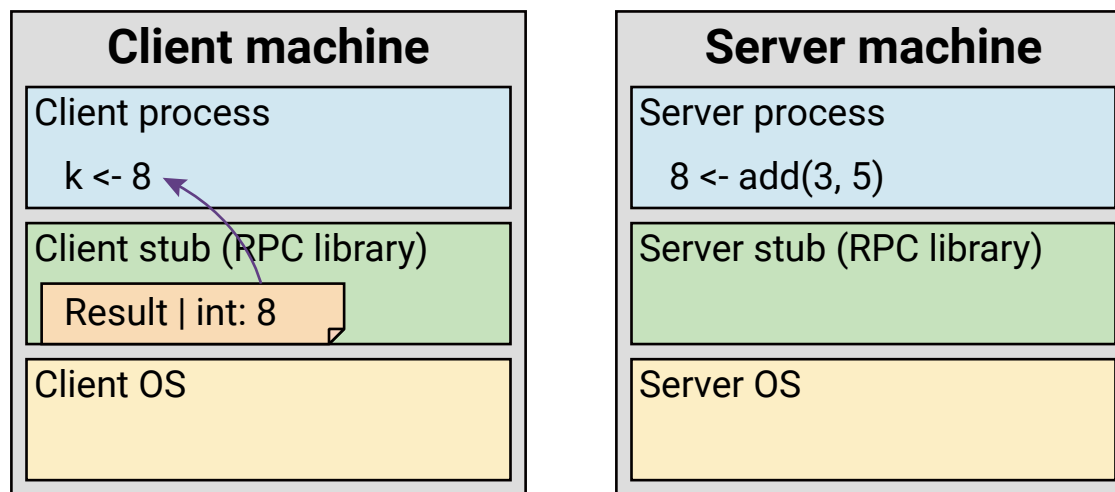




A Day in the Life of an RPC



9. Client OS receives the reply and passes up to stub
10. Client stub unmarshals return value, returns to client



What Could Possibly Go Wrong?

- Client may **crash and reboot**

What Could Possibly Go Wrong?

- Client may **crash and reboot**
- Packets may be **dropped**
 - Some individual **packet loss** in the Internet
 - **Broken routing** results in many lost packets

What Could Possibly Go Wrong?

- Client may **crash and reboot**
- Packets may be **dropped**
 - Some individual **packet loss** in the Internet
 - **Broken routing** results in many lost packets
- Server may **crash and reboot**

What Could Possibly Go Wrong?

- Client may **crash and reboot**
- Packets may be **dropped**
 - Some individual **packet loss** in the Internet
 - **Broken routing** results in many lost packets
- Server may **crash and reboot**
- Network or server might just be **very slow**

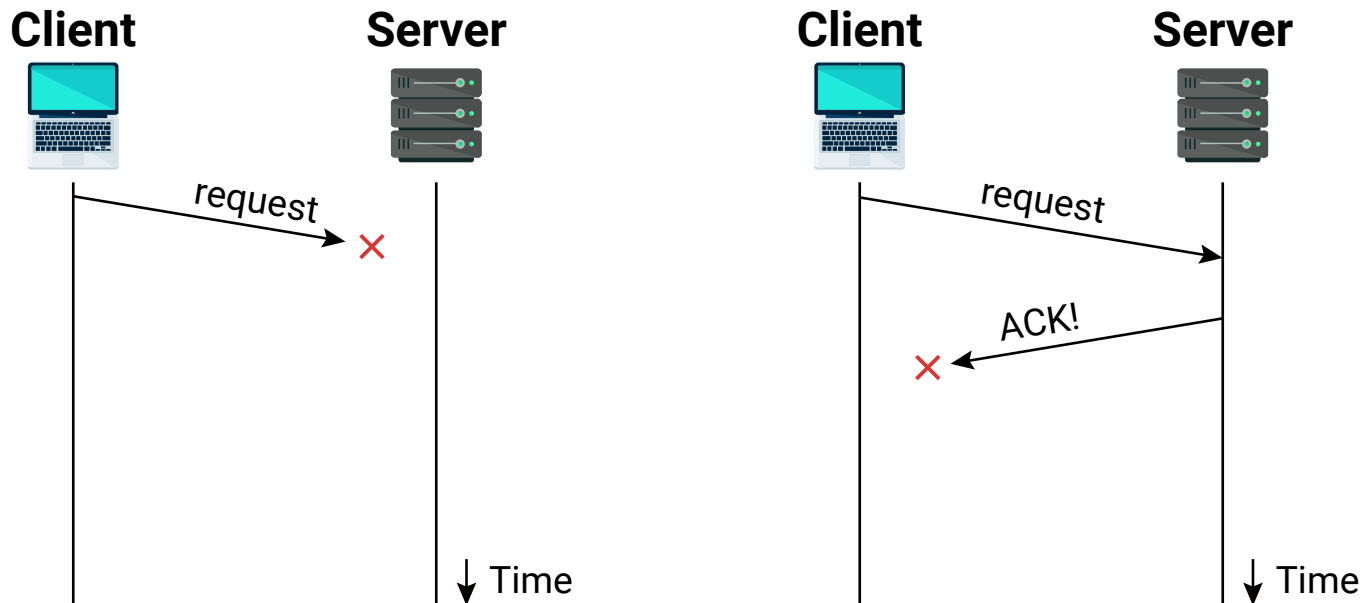
What Could Possibly Go Wrong?

- Client may **crash and reboot**
- Packets may be **dropped**
 - Some individual **packet loss** in the Internet
 - **Broken routing** results in many lost packets
- Server may **crash and reboot**
- Network or server might just be **very slow**

All of these may **look the same** to the client...



Failures, From Client's Perspective



The cause of the failure is hidden from the client!

At-Least-Once Scheme

- Simplest scheme for handling failures
1. Client stub waits for a response, for a while
 - Response is an **acknowledgement** message from the server stub

At-Least-Once Scheme

- Simplest scheme for handling failures
 1. Client stub waits for a response, for a while
 - Response is an **acknowledgement** message from the server stub
 2. If no response arrives after a fixed **timeout** time period, then client stub re-sends the request

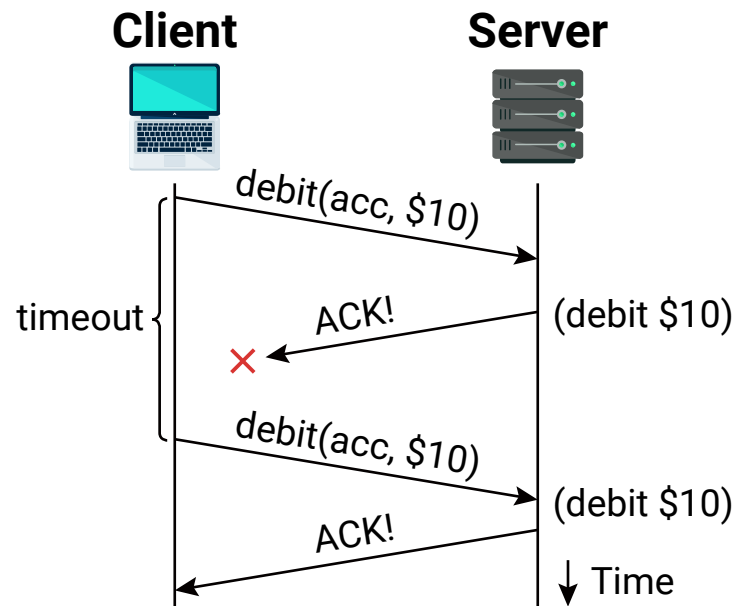
▶ At-Least-Once Scheme

- Simplest scheme for handling failures
 1. Client stub waits for a response, for a while
 - Response is an **acknowledgement** message from the server stub
 2. If no response arrives after a fixed **timeout** time period, then client stub re-sends the request
- Repeat the above a few times
 - Still no response? Return an error to the application

▶ At-Least-Once and Side Effects

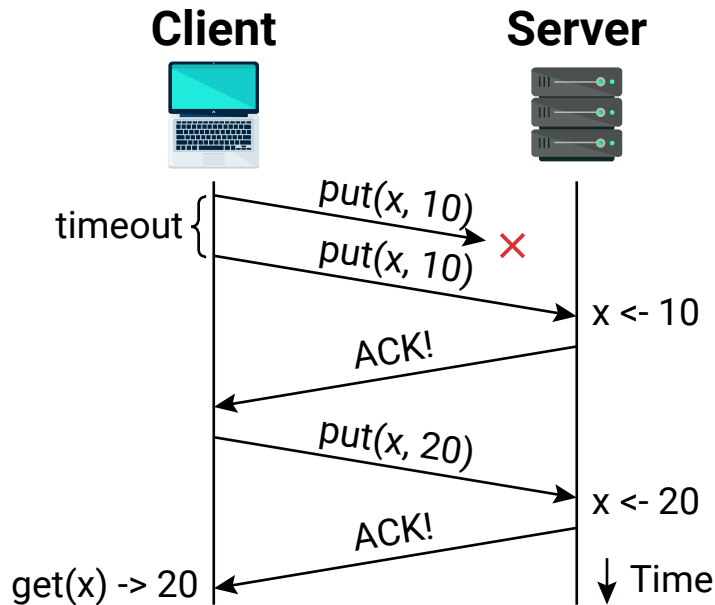
Client sends a “debit \$10 from bank account” RPC

Debit is not
idempotent!



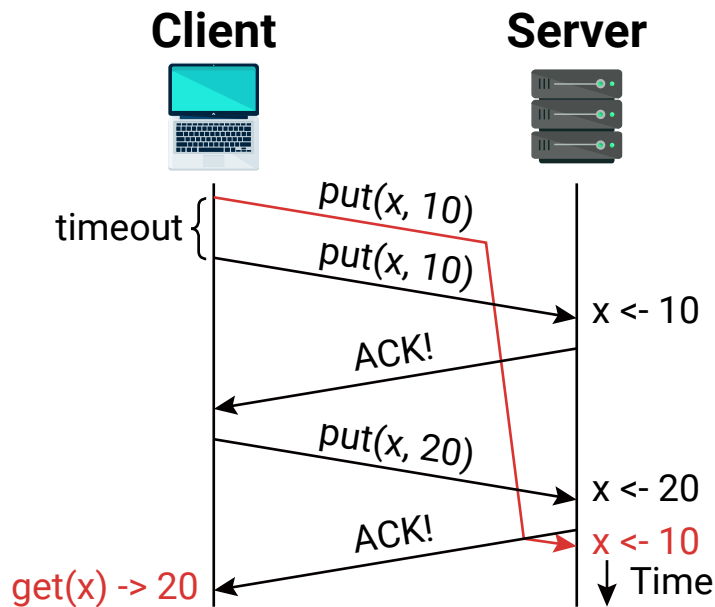
At-Least-Once and Writes

- Consider a client storing key-value pairs in a database
 - `put(x, value)`, then `get(x)`: expect answer to be `value`



At-Least-Once and Writes

- Consider a client storing key-value pairs in a database
 - `put(x, value)`, then `get(x)`: expect answer to be `value`



► So is At-Least-Once Ever Okay?

- Yes: If they are **read-only** operations with no side effects
 - e.g., read a key's value in a database
- Yes: If the application has its own functionality to cope with **duplication** and **reordering**

At-Most-Once Scheme

- Idea: server RPC stub detects duplicate requests
 - Returns previous reply instead of re-running handler

At-Most-Once Scheme

- Idea: server RPC stub detects duplicate requests
 - Returns previous reply instead of re-running handler
- How to detect a duplicate request?
 - Test: Server stub sees same function, same arguments twice

At-Most-Once Scheme

- Idea: server RPC stub detects duplicate requests
 - Returns previous reply instead of re-running handler
- How to detect a duplicate request?
 - Test: Server stub sees same function, same arguments twice
 - **No!** Sometimes applications legitimately submit the same function with same arguments, twice in a row

At-Most-Once Scheme

- How to detect a duplicate request?
 - Client stub includes unique **transaction ID** (*xid*) with each RPC request
 - Client stub uses same *xid* for retransmitted requests

```
1 # At-Most-Once Server Stub
2 if seen[xid]:
3     retval = old[xid]
4 else:
5     retval = handler()
6     old[xid] = retval
7     seen[xid] = True
8 return retval
```



At-Most-Once: Providing Unique **xids**



1. Combine a unique client ID (e.g., IP address) with the current time of day



At-Most-Once: Providing Unique **xids**



1. Combine a unique client ID (e.g., IP address) with the current time of day
2. Combine unique client ID with a sequence number



At-Most-Once: Providing Unique **xids**



1. Combine a unique client ID (e.g., IP address) with the current time of day
2. Combine unique client ID with a sequence number
3. Big random number (probabilistic, not certain guarantee)

At-Most-Once: Discarding Server State

- **Problem:** seen and old arrays will **grow without bound**
 - What is bad about the arrays growing without bound?

At-Most-Once: Discarding Server State

- **Problem:** seen and old arrays will **grow without bound**
 - What is bad about the arrays growing without bound?
- **Observation:** By construction, when the client gets a response to a particular `xid`, it will never re-send it

At-Most-Once: Discarding Server State

- **Problem:** seen and old arrays will **grow without bound**
 - What is bad about the arrays growing without bound?
- **Observation:** By construction, when the client gets a response to a particular `xid`, it will never re-send it
- Client could tell server “I’m done with `xid=x` — delete it”
 - Have to tell the server about **each and every** retired `xid`
 - Could piggyback on subsequent requests
 - How does the client know the server got the “done with `xid=x`” message?

▶ At-Most-Once: Discarding Server State

- **Problem:** seen and old arrays will **grow without bound**

At-Most-Once: Discarding Server State

- **Problem:** seen and old arrays will **grow without bound**
- Suppose `xid` = `<unique client id, sequence no.>`
 - e.g., `<42, 1000>`, `<42, 1001>`, `<42, 1002>`

▶ At-Most-Once: Discarding Server State

- **Problem:** seen and old arrays will **grow without bound**
- Suppose `xid = <unique client id, sequence no.>`
 - e.g., `<42, 1000>`, `<42, 1001>`, `<42, 1002>`
- Client includes “seen all replies $\leq X$ ” with every RPC
 - Much like TCP sequence numbers, acks
- Each one of these is cumulative: later seen messages subsume earlier ones



At-Most-Once: Concurrent Requests



- **Problem:** How to handle a duplicate request while the original is still executing?
 - Server doesn't know reply yet. And we don't want to run procedure twice



At-Most-Once: Concurrent Requests



- **Problem:** How to handle a duplicate request while the original is still executing?
 - Server doesn't know reply yet. And we don't want to run procedure twice
- **Idea:** Add a **pending** flag per executing RPC
 - Server waits for the procedure to finish, or ignores



At-Most-Once: Server Crash and Restart



- **Problem:** Server may crash and restart
- Does server need to write its arrays to disk?



At-Most-Once: Server Crash and Restart



- **Problem:** Server may crash and restart
- Does server need to write its arrays to disk?
- Yes! On server crash and restart:
 - If `old[]`, `seen[]` arrays are only in memory:
 - Server will forget, accept duplicate requests



Exactly-Once?



- Need **retransmission** of at-least-once scheme



Exactly-Once?



- Need **retransmission** of at-least-once scheme
- Plus the **duplicate filtering** of at-most-once scheme
 - To survive client crashes, client needs to record pending RPCs on disk
 - So it can replay them with the same unique identifier



Exactly-Once?



- Need **retransmission** of at-least-once scheme
- Plus the **duplicate filtering** of at-most-once scheme
 - To survive client crashes, client needs to record pending RPCs on disk
 - So it can replay them with the same unique identifier
- Plus **storage** for making server reliable
 - Even if server fails, it needs to continue with full state
 - To survive server crashes, server should log to disk results of completed RPCs (to suppress duplicates)



Exactly-Once for External Physical Actions?



- Imagine that remote operation triggers an external physical thing
 - e.g., dispense \$100 from an ATM
- ATM could crash immediately before or after dispensing
 - Don't know which one happened
 - Operation is not atomic (logging and dispensing are separate actions)
- **Can't achieve exactly-once in general**, in presence of external actions

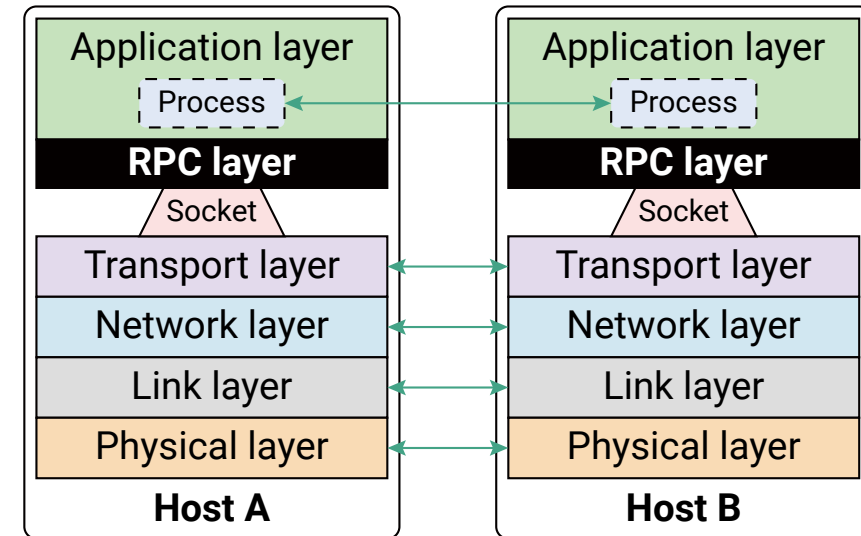
Exactly-Once for Digital Systems

- Digital systems don't have this issue because they follow three steps:
 1. Log intention to perform action
 2. Perform action
 3. Commit action — no take-backs after this point
- If crash happens before step 3, the action can be **reverted**.

The issue with external actions is that actions can't be reversed — the ATM can't take back the \$100 after it's been dispensed.

Summary: RPCs and Network Communication

- Layers are our friends!
- RPCs are everywhere
- Help support machine heterogeneity
- Subtle issues around failures
 - At-least-once w/ retransmission
 - At-most-once w/ duplicate filtering
 - Discard server state w/ cumulative acks
 - Exactly-once with:
 - at-least-once + at-most-once + fault tolerance + no external actions





© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.